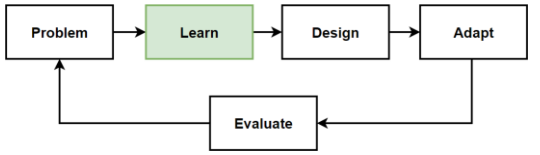
CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

MINH HỌA ỨNG DỤNG WEBSITE THƯƠNG MẠI ĐIỆN TỬ VỚI KIẾN TRÚC MICROSERVICE

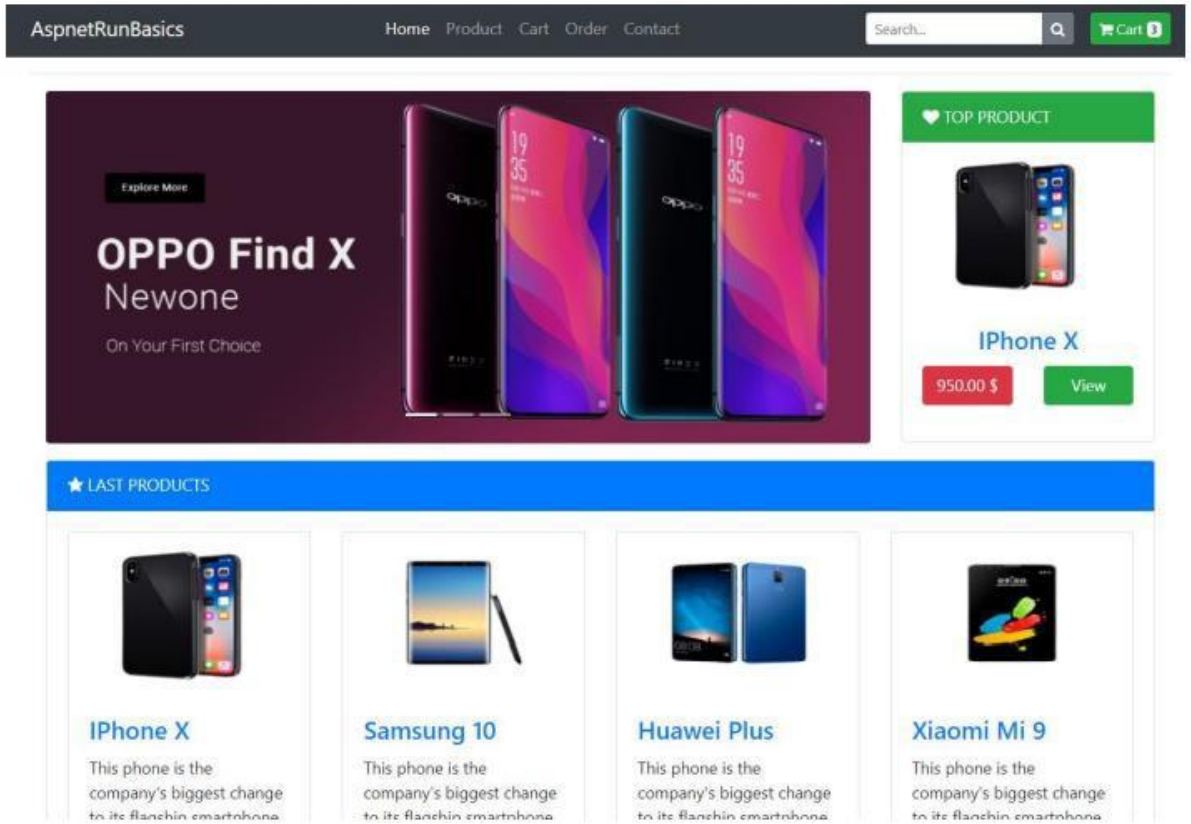
Microservices on .Net platforms which used Asp.Net Web API, Docker, RabbitMQ, MassTransit, Grpc, Ocelot API Gateway, MongoDB, Redis, PostgreSQL, SqlServer, Dapper, Entity Framework Core, CQRS and Clean Architecture implementation.
Centralized Distributed Logging with Elasticsearch, Kibana and SeriLog, use the HealthChecks with Watchdog, Implement Retry and Circuit Breaker patterns with Polly.

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Implementation of Microservices Architecture



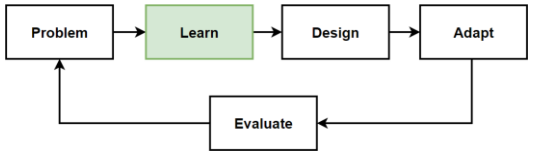
- Clone Microservices Repository
- Docker-compose up
- Follow steps on github documentation
- Run the Project Section
- DEMO



https://github.com/aspnetrun/run-aspnetcore-microservices

Mehmet Ozkaya

Big Picture of Microservices Architecture



Client Apps

WebApp
AspnetRunBasics

WebSPA
TBD

Api Gateways / BFF

OcelotApiGw

Aggregator

WebStatus
HealthChecks

Microservices

Catalog.API
MongoDB

Basket.API
Redis

Discount.Grpc
PostgreSQL

Ordering.API
SQL Server

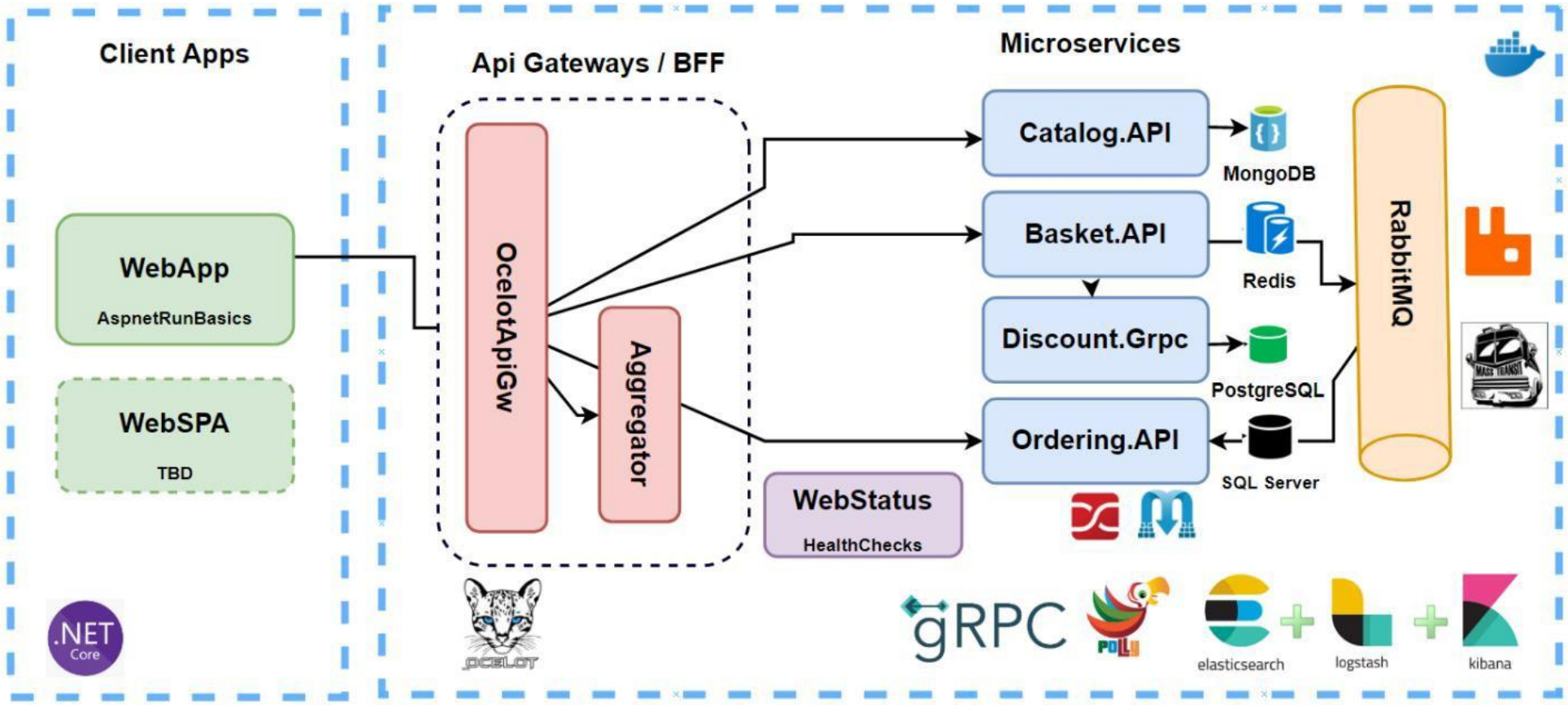
RabbitMQ

.NET Core

gRPC

elasticsearch + logstash + kibana

DEMO: Microservices Architecture Code Review



DEMO: Code review of Microservices Architecture .NET Implementation

- https://github.com/aspnetrun/run-aspnetcore-microservices
- https://github1s.com/aspnetrun/run-aspnetcore-microservices

Alternative Link - https://github.com/mehmetozkaya/MicroservicesApp

CoolStore Web Application

Modern Application on Dapr and Tye

CoolStore Website is a containerised microservices application consisting of services based on .NET Core running on Dapr. It demonstrates how to wire up small microservices into a larger application using microservice architectural principals.

https://github.com/vietnam-devs/coolstore-microservices

Introduction

CoolStore Website is a containerised polyglot microservices application consisting of services based on .NET Core, NodeJS and more running on Service Mesh. It demonstrates how to wire up small microservices and build up a larger application using microservice architectural principals.

It's mainly building for .NET ecosystem with a lot of popular libraries and toolkits which have used by .NET community for a long time. Additionaly, it uses and experiements new components and libraries to build the modern application with cloud-native apps approach.

Bussiness Context

CoolStore Website has the basic business scenario for **Product Catalog**, **Shopping Cart**, **Payment Process**, **Inventory**, **Rating**, and **Access Control**.

With **Product Catalog**, **Buyer** can browse the **products list** with supported filtering and sorting by product name and price functionalities. And she can see the **detail of the product** on the product list page by clicking on it, in the detail page, she can see a name, description, available product in the **inventory**, the inventory store information like stock address and location, a **hot product** flag (if has) and **rating**. **SysAdmin** in the system can **manage the product** and has the ability to **assign this product to the existed inventory**.

With **Shopping Cart**, **Buyer** can **buy** any **product** on the **product list** via the Buy button on any product. Besides, she can **buy** the product in the **detail product page**. After bought product, she should see these products in the shopping cart and the **summary panel** with basic information such as cart total cost, promotion item saving cost, subtotal cost, shipping cost, promotion shipping savings cost, total order amount. And whenever she **buy more products** or **remove some products** out of the cart, then the **summary panel** and **shopping cart** should be **updated**. After all, she can do a **checkout process** to pay money by clicking the Check Out button on the shopping cart page. **SysAdmin** can **see all the shopping cart of any user** so that he can **enable or disable** any invalid shopping cart in CoolStore website.

With **Payment process**, after Buyer clicked checkout button, the system will start to **validate the product** information, **process payment**, and then **send an email** to **Buyer** so that she knows about what is going on.

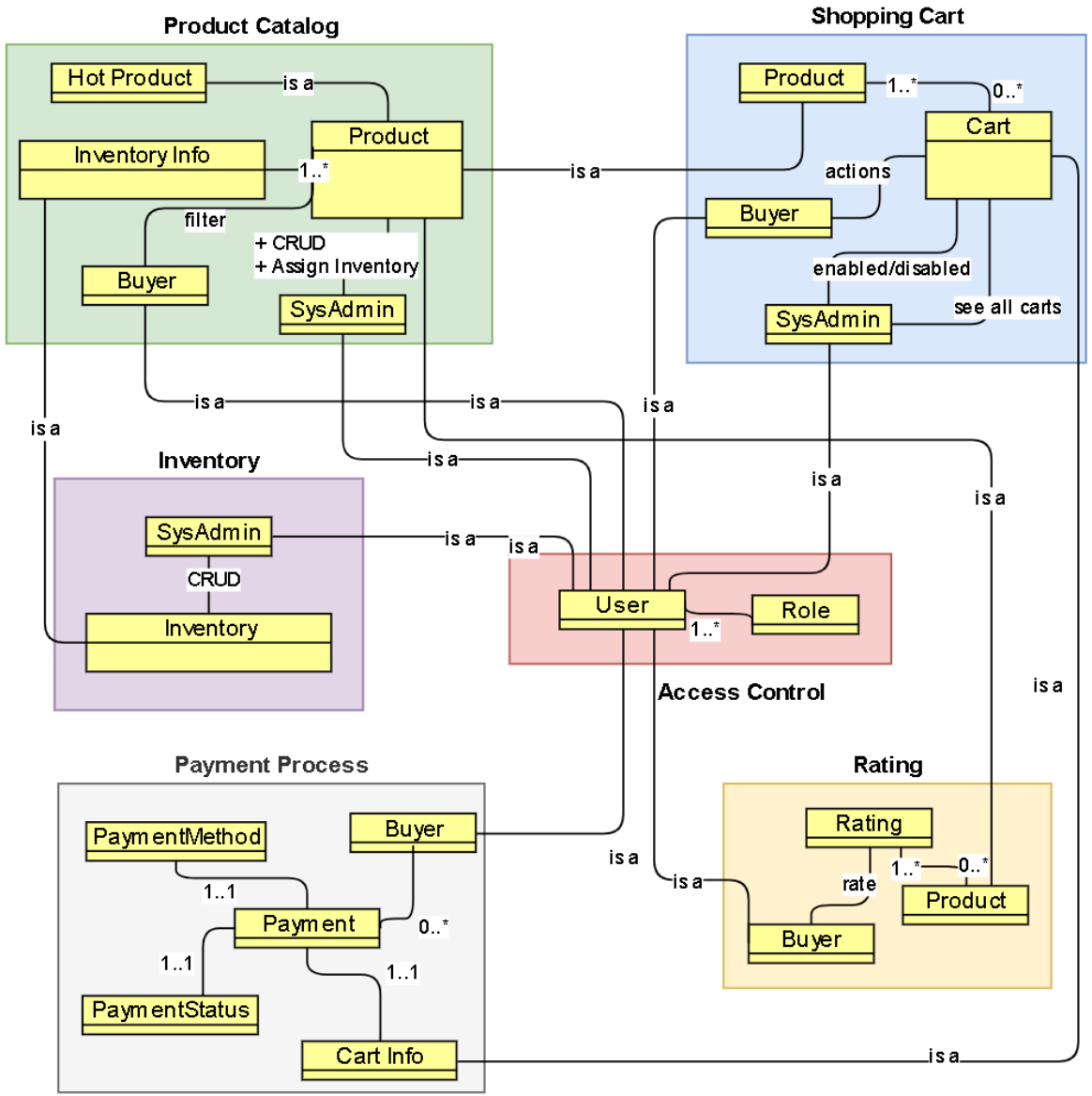
With **Inventory**, **SysAdmin** can **manage the inventory**.

With **Rating**, **Buyer** can **rate any product** that she thinks is good (1 -> 5 stars).

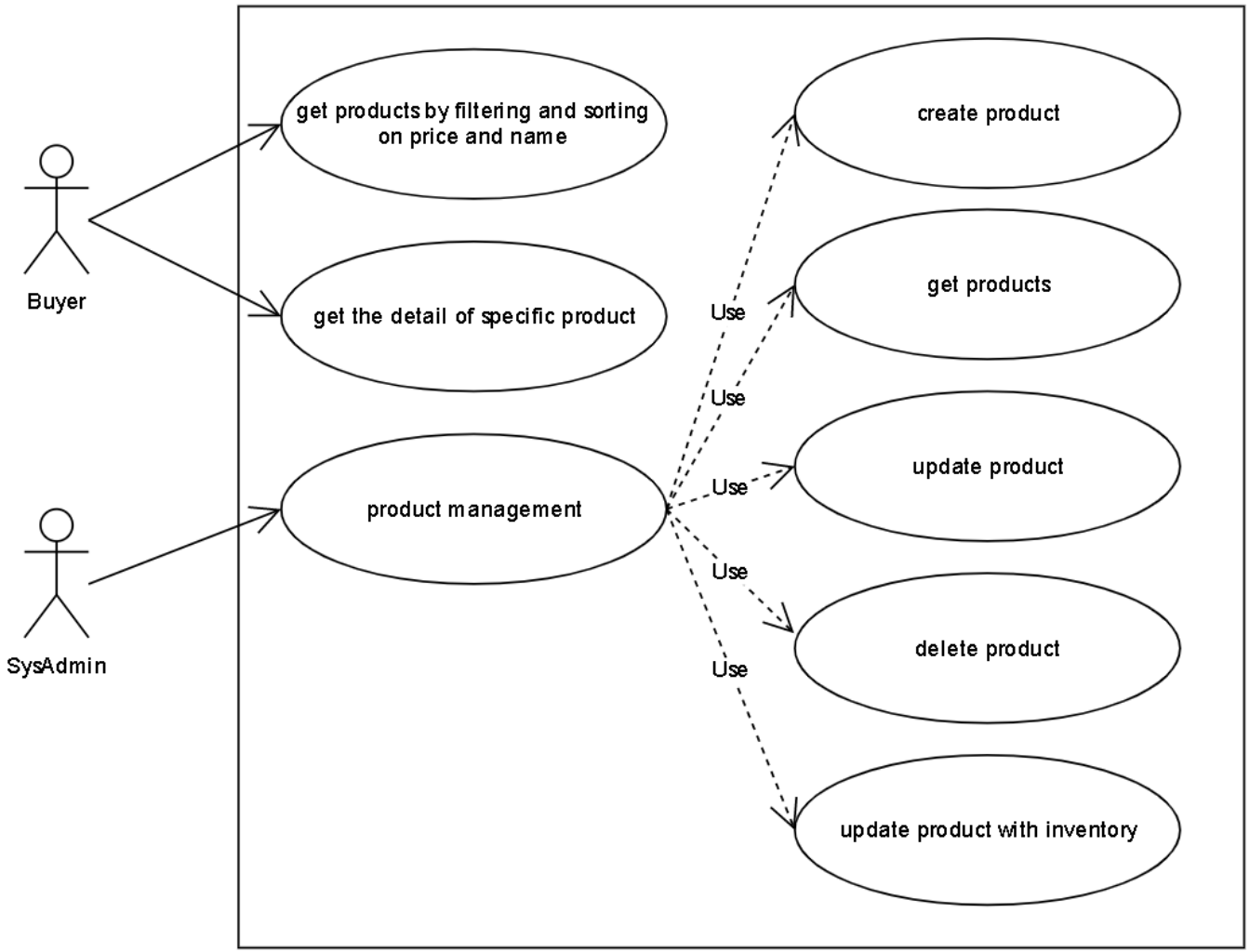
With **Access Control**, **Buyer** or **SysAdmin** can **log on/off the system** and if **Buyer**, she will be brought into the **product catalog page**, and if **SysAdmin**, he will be brought into the **administration page**.

Some of the **one-off tasks** need to be done when CoolStore website starts such as **SysAdmin user, two Buyer users** and **sample data** for **product**, **inventory**, **rating** for a **few products**.

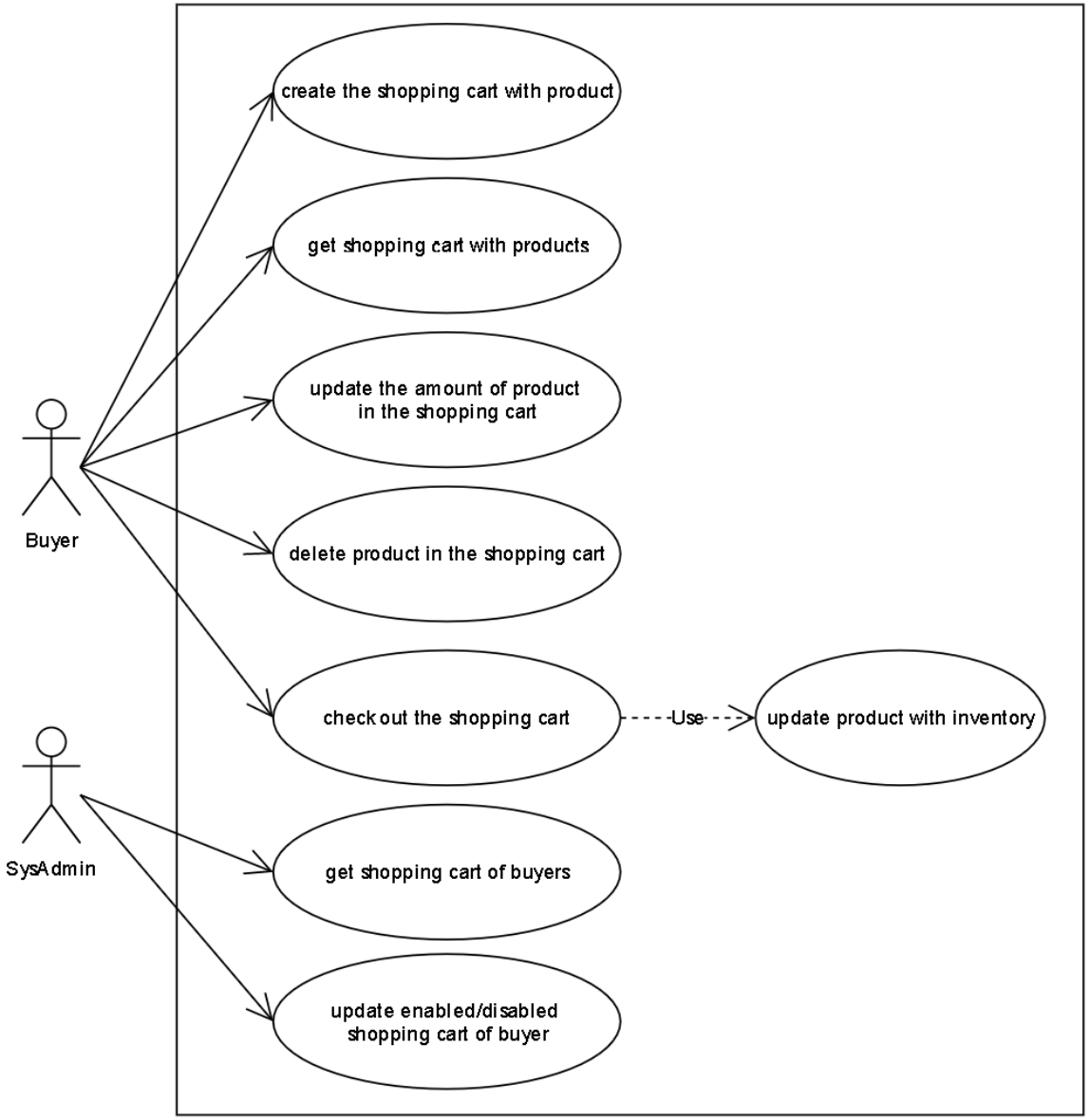
Conceptual Model



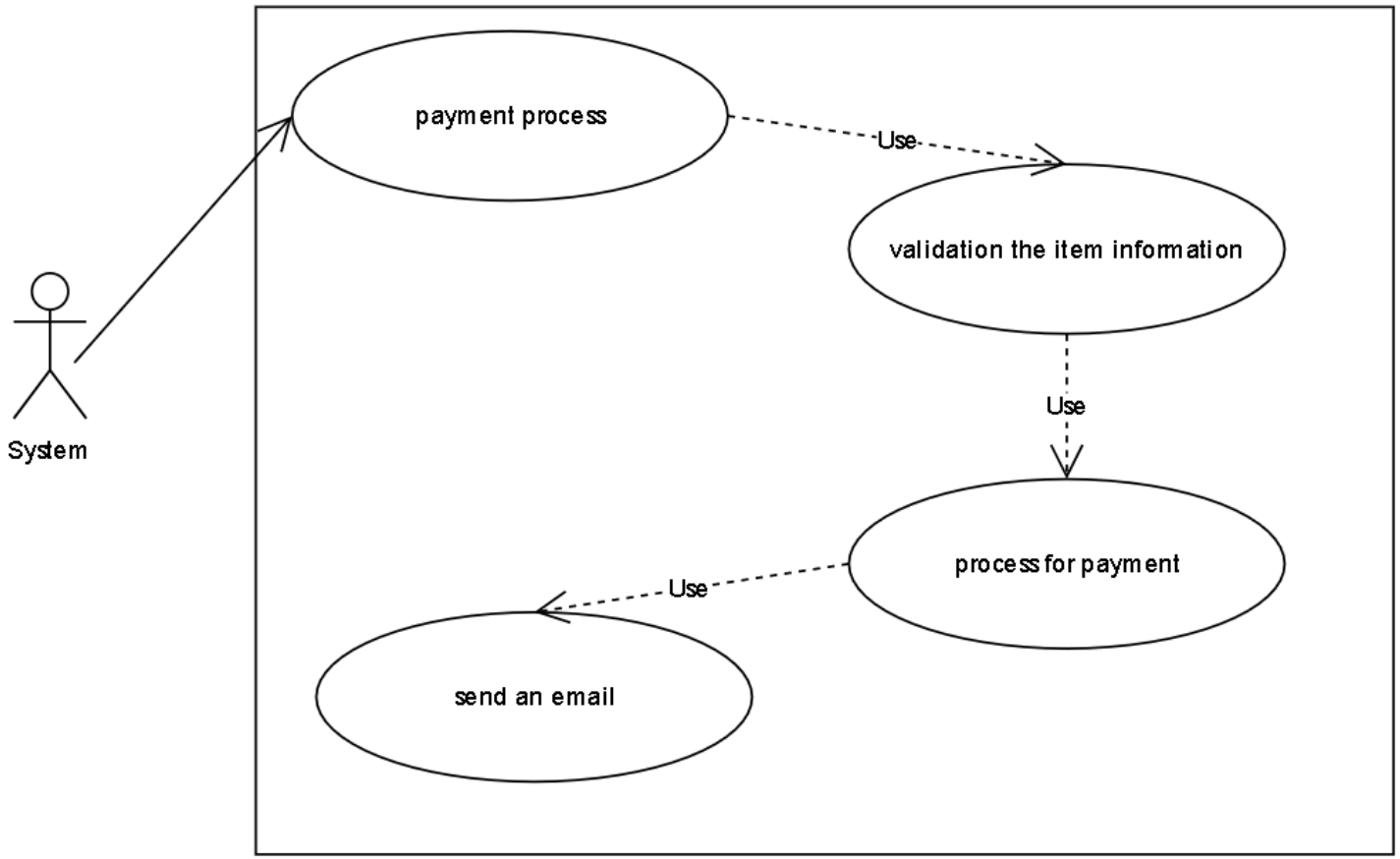
Use Case: Product Catalog



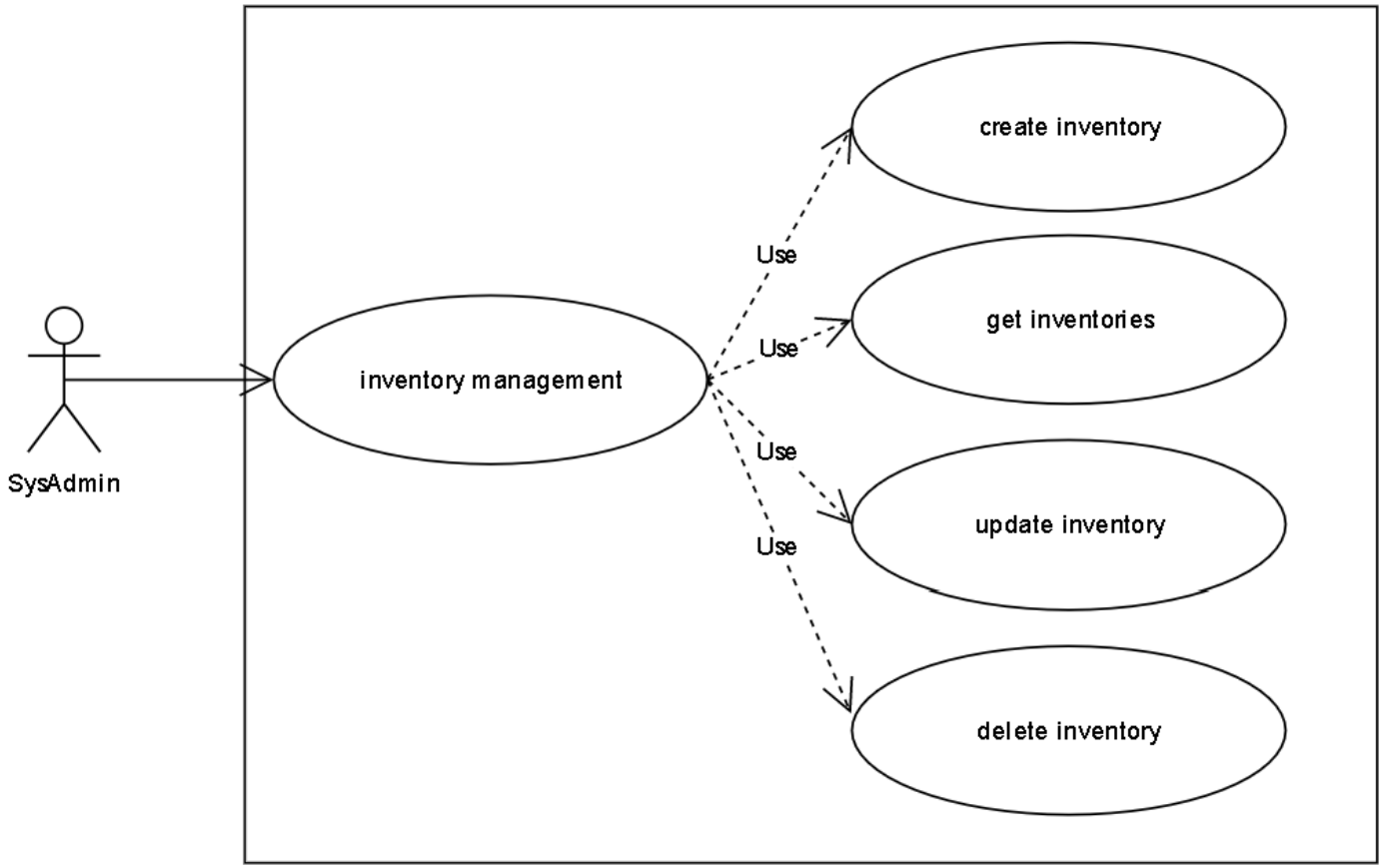
Use Case: Shopping Cart



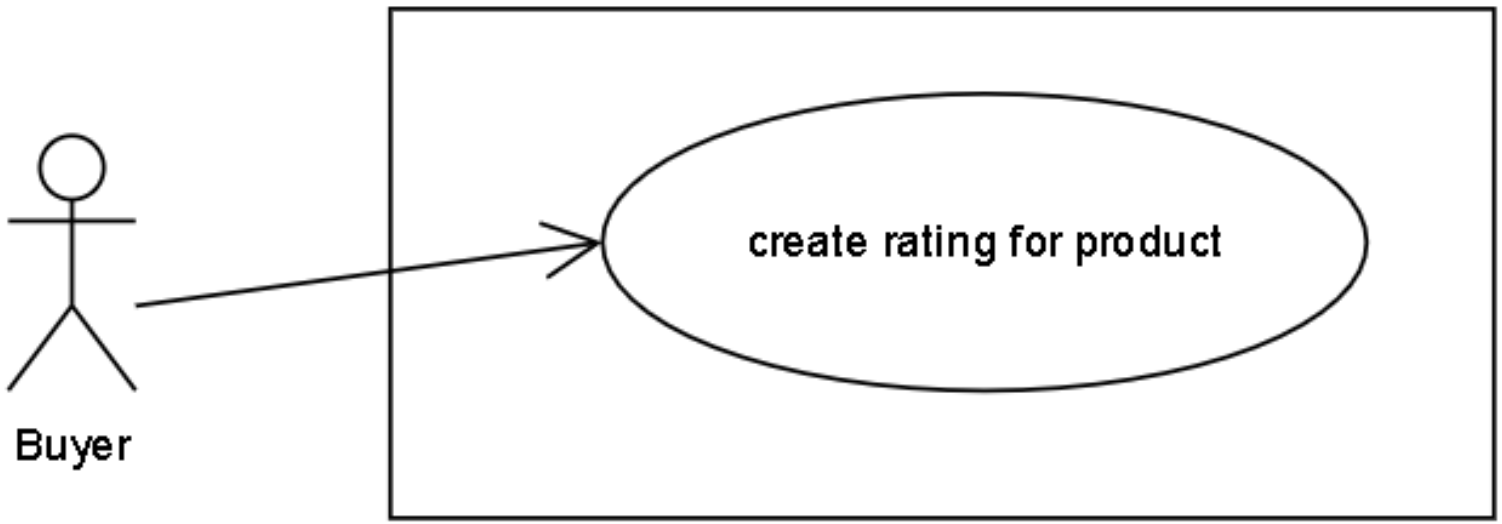
Use Case: Payment Process



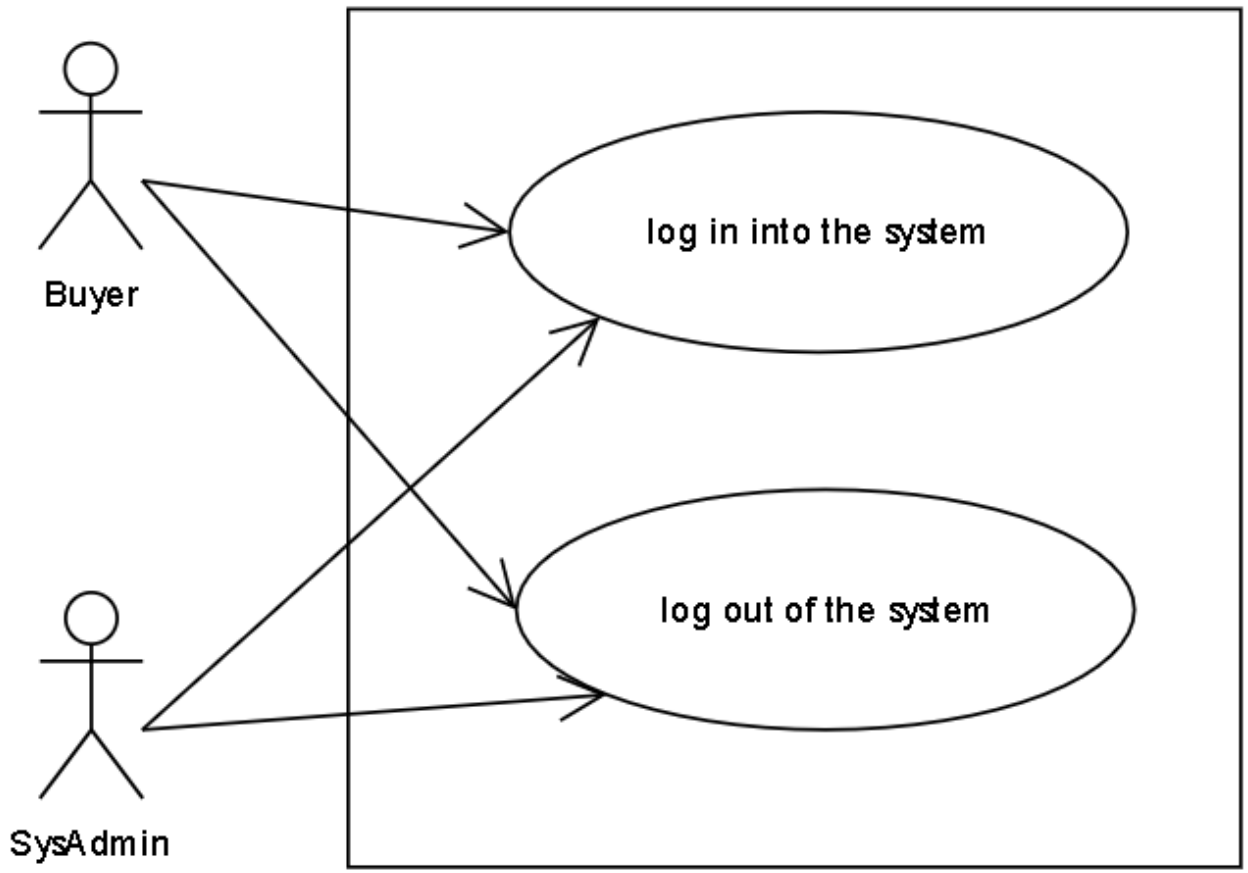
Use Case: Inventory



Use Case: Rating



Use Case: Access Control



User Story

Product Catalog

- As a **Buyer**, I want to **see the list of products with filtering, sorting on the home page** (name, photo, short description, rating, and hot product flag which is a product with a lot of people see or buy).
  - Whenever filtering with any price and name of the product, then the list of products need to narrow down with appropriate products.
  - Whenever sorting with descending or ascending on the price or name of the product, then the list of products need to follow with this sorting.
  - Whenever both filtering and sorting in action, then the list of products shall be effective by both of description above.
- As a **Buyer**, I want to **navigate into the detail of one product** with the basic attributes such as name, description, available product in the inventory, the inventory store information like stock address and location, a hot product flag (if has) and rating.
- As a **SysAdmin**, I want to **manage a product** (CRUD actions) and **assign one existing inventory into the product**.

User Story

Shopping Cart

- As a **Buyer**, I want to **buy any product on the product catalog page** (add this product into the shopping cart - one product will be added by default).
- As a **Buyer**, I want to **see the product detail and buy this product** if I like (add this product into the shopping cart - one product will be added by default).
- As a **Buyer**, I want to **see the list of products** I just added into the shopping cart, and I would like to **see the summary information panel** for the current shopping cart such as cart total cost, promotion item saving cost, subtotal cost, shipping cost, promotion shipping savings cost, total order amount on this page.
- As a **Buyer**, I want to **update the amount of product in the shopping cart**.
  - Whenever updating amount of product happens, then the summary information panel needs to be updated accordingly to changes.
- As a **Buyer**, I want to **delete any product in the shopping cart** which they don't want to buy anymore.
  - Whenever the deleting amount of product happens, then the summary information panel needs to be updated accordingly to changes.
- As a **Buyer**, I want to do **check out my shopping cart**.
  - Whenever the number of products in the shopping cart is zero then this checks out process does not happen.
- While a shopping cart was checked out, the payment process starts.
- As a **SysAdmin**, I want to **see shopping cart of all buyers with information** about cart total cost, promotion item saving cost, subtotal cost, shipping cost, promotion shipping savings cost, total order amount.
- As a **SysAdmin**, I want to **enable/disable any shopping cart of any buyer**.

User Story

Payment Process

- Any Buyer can do the payment.
- In the moment of the **payment process** happening, we start to **validation the item information**, **process for payment** and subsequently **send an email to the Buyer** (because this is just the demo so we don't actually integrate with the payment gateway)
  - Whenever any product information is invalid, then the payment process will be canceled and one email will be sent to Buyer for notification.
  - Whenever ending this payment process, we mark the payment of this cart is processed status and send an email to let Buyer knows.

Inventory

- As a **SysAdmin**, I want to **manage inventory** (CRUD actions).

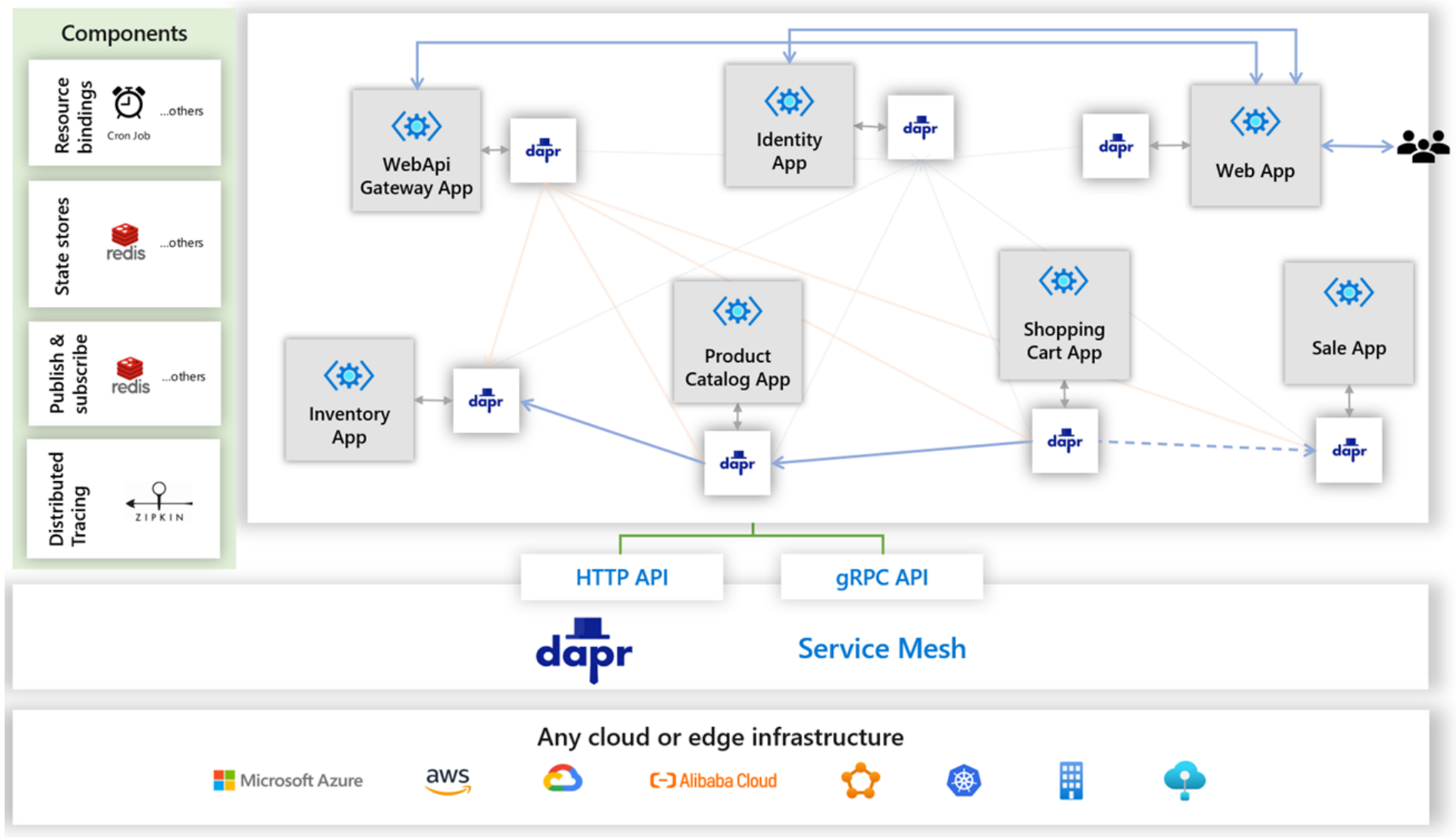
Rating

- As a **Buyer**, I want to **rate for each product** that I think is good (1 -> 5 stars).

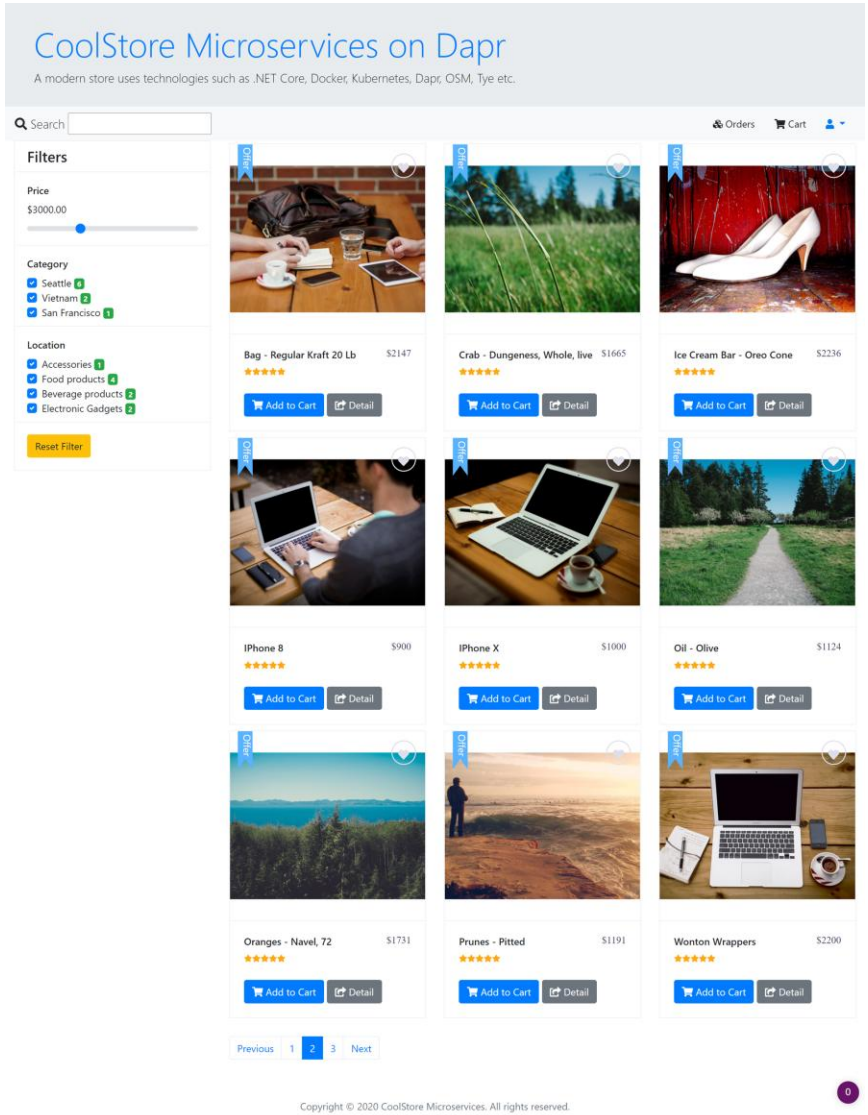
Access Control

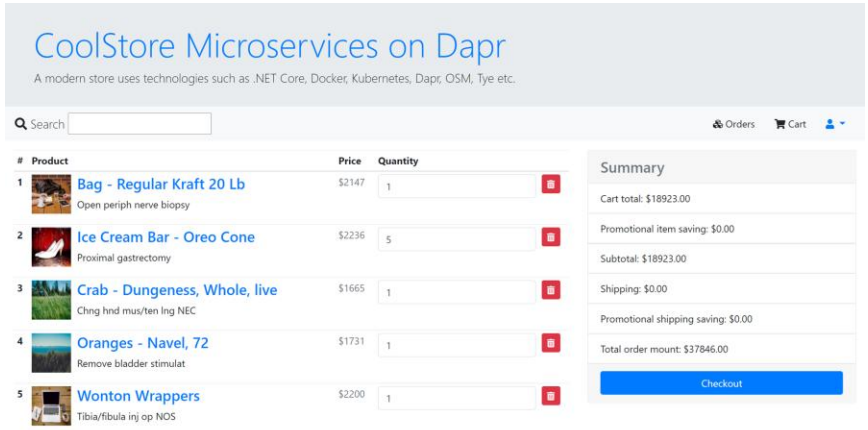
- Each Buyer/SysAdmin is a User.
- As a **Buyer/SysAdmin**, I want to **log-in to the system**.
  - Whenever a user with a Buyer role does a login, then I will be brought to the product catalog page.
  - Whenever a user with a SysAdmin role does a login, then I will be brought to the administration page.
- As a **Buyer/SysAdmin**, I want to **log-out of the system**.

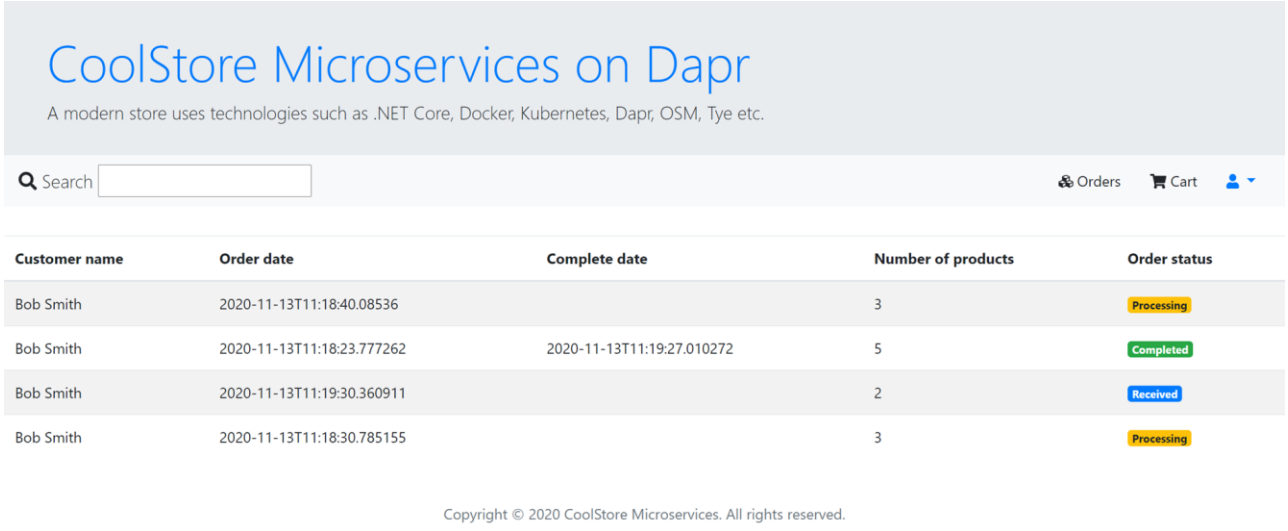
Dapr Building Blocks



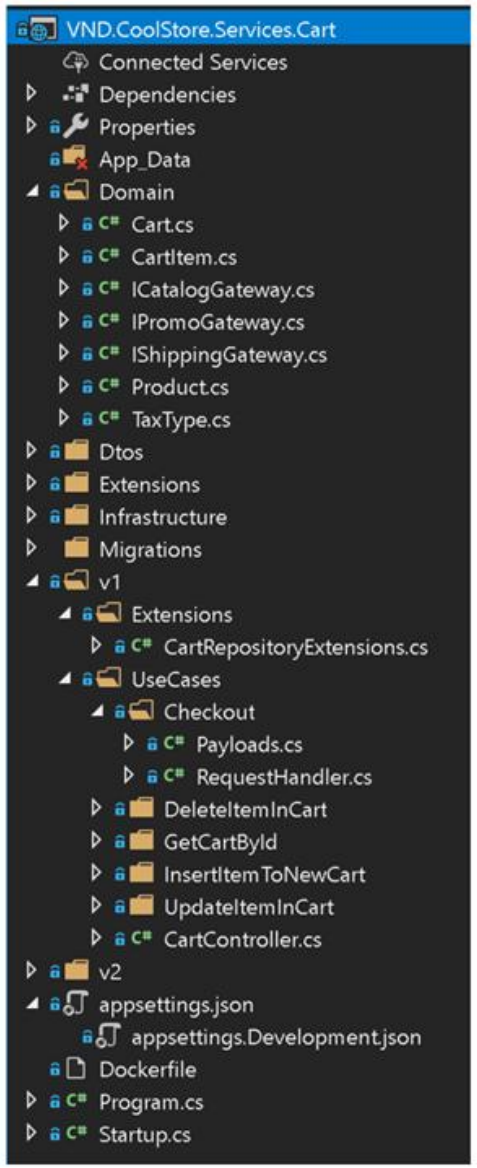
Screenshots



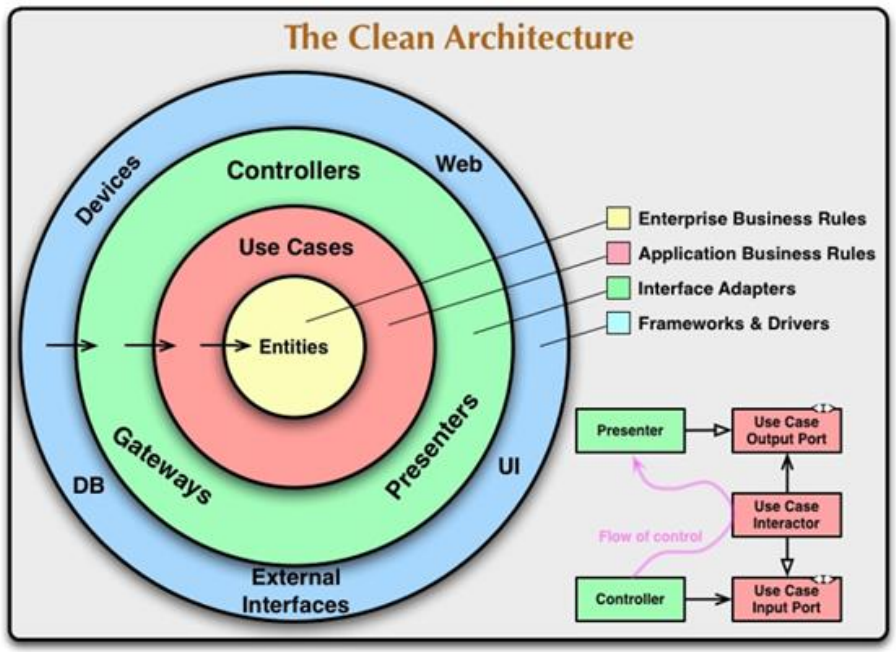




µService development

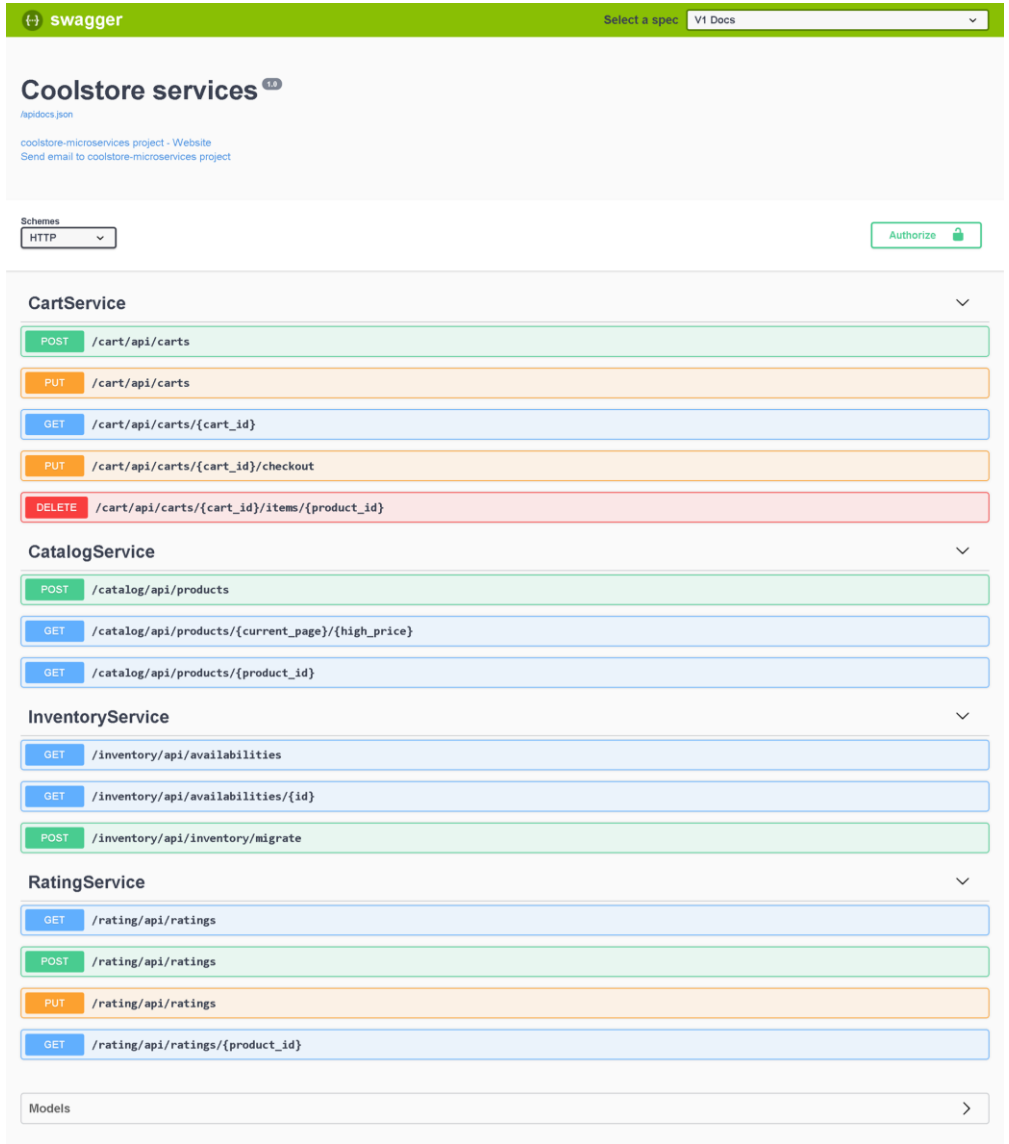


Clean Domain-Driven Design

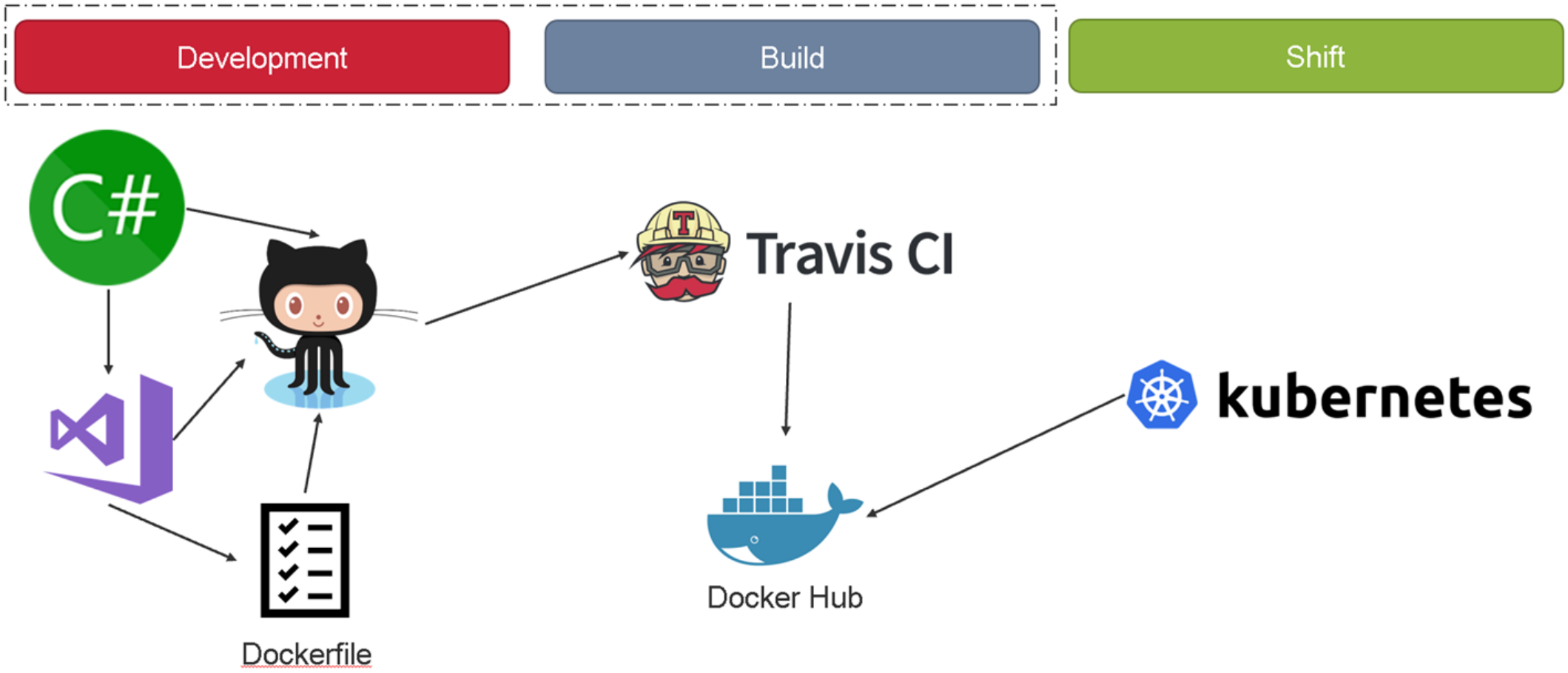


https://github.com/vietnam-devs/coolstore-microservices

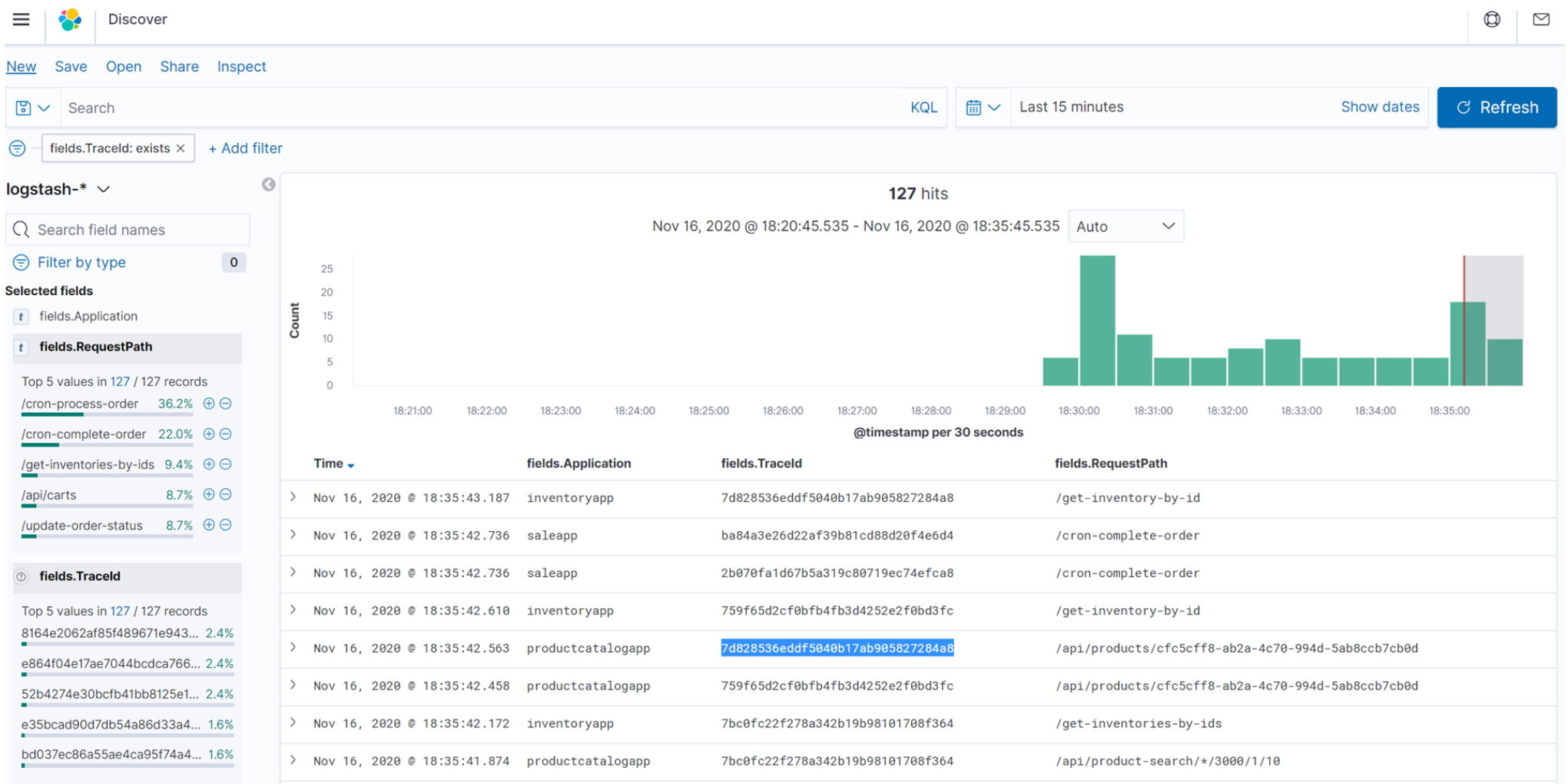
API Design


swagger
Select a spec
V1 Docs
Coolstore services
1.0
/apidocs.json
coolstore-microservices project - Website
Send email to coolstore-microservices project
Schemes
HTTP
Authorize
CartService
POST /cart/api/carts
PUT /cart/api/carts
GET /cart/api/carts/{cart_id}
PUT /cart/api/carts/{cart_id}/checkout
DELETE /cart/api/carts/{cart_id}/items/{product_id}
CatalogService
POST /catalog/api/products
GET /catalog/api/products/{current_page}/{high_price}
GET /catalog/api/products/{product_id}
InventoryService
GET /inventory/api/availabilities
GET /inventory/api/availabilities/{id}
POST /inventory/api/inventory/migrate
RatingService
GET /rating/api/ratings
POST /rating/api/ratings
PUT /rating/api/ratings
GET /rating/api/ratings/{product_id}
Models

CI/CD


Development
Build
Shift
C#
Travis CI
kubernetes
Docker Hub
Dockerfile

Debug and Tracing Apps



CÁM ƠN ĐÃ CHÚ Ý LẮNG NGHE!